

Relatório

Alunos:

Cauã Fernandes

Davi Rodrigues

Rafael Moraes

Trie

A Trie foi implementada com um alfabeto de 36 caracteres: as 26 letras minúsculas do alfabeto latino (a–z) e os 10 dígitos decimais (0–9). Cada nó da estrutura contém um array de 36 ponteiros para seus filhos, onde cada posição corresponde a um caractere válido.

O mapeamento de caractere para índice é feito pelo método `charToIndex`:

- letras: índice a = 0, b = 1, ..., z = 25
- dígitos: índice 0 = 26, 1 = 27, ..., 9 = 35

Antes de qualquer inserção ou busca, títulos e prefixos são convertidos para uma chave de busca pelo método `toSearchKey`. Esse método percorre o texto caractere por caractere, descartando espaços e convertendo letras maiúsculas para minúsculas. Assim, "Half Life", "halflife" e "HALF L" produzem a mesma chave "halflife", garantindo buscas case-insensitive que ignoram espaços.

Métodos e custos

insert

O título do jogo é convertido para chave de busca. Em seguida, a Trie é percorrida caractere por caractere: se o filho correspondente ao caractere atual não existe, um novo nó é criado; caso contrário, apenas avança. Ao final do título, o último nó recebe `isEndOfTitle = true` e um ponteiro para o objeto `Game` já existente no array do catálogo — nenhuma cópia do jogo é criada.

O custo de insert é $O(L)$, pois percorre um caminho de comprimento L na Trie, criando nós quando necessário.

contains

Segue a mesma lógica de navegação do insert, mas apenas lê a estrutura. Se em qualquer ponto o filho necessário for `nullptr`, retorna `false`. Chegando ao fim da chave, retorna o valor de `isEndOfTitle`, isso distingue um prefixo de um título completo armazenado.

O custo de contains é $O(L)$ também, pois percorre o mesmo caminho, apenas lendo.

autocomplete

Primeiro verifica se $k \leq 0$ e retorna vazio. Depois converte o prefixo para chave de busca e navega na Trie até o nó correspondente ao último caractere do prefixo. Se o caminho não existir, retorna vazio. Do nó encontrado em diante, executa uma busca em profundidade (DFS) pelo método auxiliar `collectGames`, que visita toda a subárvore e coleta ponteiros de todos os jogos cujos títulos começam com o prefixo. O vetor resultante é então ordenado por `sortResults` e truncado em k elementos.

O custo do autocomplete é $O(p + s + r \log r)$, sendo $O(p)$ para navegar até o prefixo, $O(s)$ para o DFS na subárvore, $O(r \log r)$ para o merge sort implementado por `sortResults`.

sortResults

Implementa merge sort manualmente (sem `std::sort`). O critério de comparação é: maior popularidade primeiro; em empate, ordem alfabética crescente pela chave de busca do título.

O custo do `sortResults` tem custo $O(r \log r)$ como dito também no anterior.

Comparação de Memória

Trie vs. solução ingênua

Uma solução simples percorreria o array completo de N jogos a cada busca, verificando se cada título começa com o prefixo — custo $O(N \times L)$ por consulta. Com a Trie, a busca vai diretamente ao nó do prefixo em $O(p)$ e explora apenas a subárvore correspondente em $O(s)$, onde $s \ll N \times L$ quando o prefixo é específico. A vantagem cresce com o tamanho do catálogo.

Memória

A Trie consome mais memória do que um array simples. Cada nó alocar um array fixo de 36 ponteiros, independente de quantos filhos realmente usa. Para N jogos e títulos de comprimento médio L , o número de nós no pior caso é $O(N \times L)$.

Compartilhamento de prefixos

Títulos com prefixos comuns compartilham os mesmos nós iniciais. Por exemplo, "hades", "halo" e "half-life" compartilham os nós `h` e `a`, criando 14 nós ao invés de 16 (se fossem caminhos independentes). Catálogos com muitos títulos semelhantes se beneficiam mais desse compartilhamento.

Impacto do tamanho do alfabeto

Cada nó reserva ALPHABET_SIZE ponteiros, independente de quantos são nullptr. Com alfabeto de 36 caracteres, cada nó usa 36×8 bytes (em sistemas 64-bit) = 288 bytes. Um alfabeto menor reduziria esse custo por nó, mas limitaria os caracteres aceitos; um alfabeto maior (ex.: incluindo acentos) aumentaria o consumo sem benefício para este problema, já que os testes não usam acentos.